

Erfahrungsbericht: Einsatz von LLMs zur Portierung und Dokumentation von Legacy-Code

Quelle: Arbeitsgruppe „GenAI im Aktuariat“ der Deutschen Aktuarvereinigung (DAV)

1. Ausgangslage

In vielen Versicherungshäusern existieren Alt-Systeme, deren Ursprung teilweise Jahrzehnte zurückliegt. Diese Legacy-Anwendungen sind in Programmiersprachen wie COBOL, PL/I, FORTRAN oder ASSEMBLER implementiert, oder sie liegen in Form komplexer Excel-Modelle vor. Häufig ist das Wissen über die Funktionsweise dieser Systeme stark personengebunden – mit dem Ausscheiden langjähriger Entwickler und Aktuare droht dieses Know-how verloren zu gehen.

Traditionell standen Unternehmen damit vor zwei Optionen:

1. Weiterbetrieb der Legacy-Systeme bei gleichzeitig steigendem Aufwand für Wartung und Know-how-Sicherung.
2. Neuentwicklung der Anwendungen und Migration der alten Daten mit hohem Projekt- und Ressourcenaufwand.

Die Reife von Large Language Models (LLMs) eröffnet nun potenziell eine dritte Möglichkeit: automatisierte bzw. halbautomatisierte Dokumentation und/oder Portierung von Legacy-Code in moderne Programmierparadigmen. In diesem Use Case sollte verprobt werden, ob und wie weit der Stand der Technik das ermöglicht.

2. Mögliche Anwendungsfelder

2.1 Excel-Modelle

Ein Anwendungsfeld sind komplexe Excel-Rechner, die nur noch von wenigen Personen verstanden werden. Durch eine Übersetzung nach Python lässt sich der Funktionsumfang professionalisieren:

- bessere Testbarkeit und Wartbarkeit,
- Nutzung von Versionskontrolle (z. B. GitHub),
- Einsatz moderner Entwicklungswerkzeuge und Qualitätssicherung.

In der DAV-Arbeitsgruppe Bestandsmanagement wurde dies am Beispiel eines Leben-Tarifrechners der Pfefferminzia erfolgreich umgesetzt. Dabei wurden zwei Ansätze verfolgt –

einer stärker „hands-on“ mit direkter Übertragung, der andere stärker industrialisiert mit modularer Strukturierung und Tests. Die Portierung nach Python war erfolgreich.

Details befinden sich auf Github im Bereich der Deutschen Aktuarvereinigung:
<https://github.com/DeutscheAktuarvereinigung/portxlpv>

2.2 Klassische Legacy-Systeme

Bei großvolumigen, historisch gewachsenen Kernsystemen kamen für eine Portierung der Vergangenheit Transpiler-Lösungen zur Anwendung (z. B. COBOL → Java). Sie erzeugen Code in neuer Sprache, behalten aber in der Regel weitgehend die alte Logik und Struktur bei („Jobol“). Ergebnis: oft eingeschränkte Wartbarkeit und Fehler durch falsche Übersetzung.

Im Use Case verprobt wurde eine Neucodierung mit Nutzung des Legacy-Codes als Spezifikation bzw. Vorbild. Hier dient der alte Code als „single source of truth“. Mit Unterstützung durch LLMs sollte sich daraus Code in moderner Sprache erzeugen lassen, der zugleich die Paradigmen der Zielsprache (z. B. objektorientierte Strukturen in Java, modulare Architekturen in Python) berücksichtigt.

2.3 Exkurs: Gängige Werkzeuge für die Code-Übersetzung im Überblick

Die Übersetzung von Programmiersprachen hat die besondere Herausforderung, dass zwar die Grammatik der Programmiersprachen vergleichsweise einfach ist, jedoch im Gegensatz zu menschlicher Sprache keine Toleranz bei der Ausgabe besteht: der ausgegebene Code kann nur dann kompiliert werden, wenn er vollständig fehlerfrei ist.

Gängige Werkzeuge der Code-Übersetzung lassen sich grob in drei Gruppen einteilen: a) Source-to-Source-Compiler (Transpiler), eine länger bekannte regelbasierte Technologie; b) spezialisierte Übersetzer basierend auf neuronalen Netzen; und c) LLM-basierte Ansätze.

a) Ein Transpiler ist ein Compiler, der Quellcode aus einer Programmiersprache in eine andere, zumeist eng verwandte Programmiersprache übersetzt.

Ein typische Beispiel ist *Cfront* aus dem Jahr 1983: In den Anfangszügen der Entwicklung von C++ gab es noch keinen eigenständigen Compiler für diese Programmiersprache. Daher wurde mit *Cfront* ein Transpiler erstellt, der Quellcode in C++ in C-Code übertrug, welcher dann in einem existierenden C-Compiler in ausführbaren Code übersetzt wurde.

Wesentliche Eigenschaften von Transpilern sind ([1]):

- Transpiler sind grundsätzlich deterministisch, daher ist die Übersetzung leicht nachvollziehbar;
- Auch große Codestücke können übersetzt werden, es gibt kein Kontextfenster, das nur eine begrenzte Anzahl an Tokens aufnehmen kann;
- Jedoch erzeugen LLM-basierte Methoden Code, der eher wie menschlich erzeugt aussieht und daher natürlicher erscheint

b) Auch Methoden des statistischen Lernens, insbes. neuronale Netze, können zur Übersetzung von Code eingesetzt werden. Die Modelle wurden ursprünglich auf

Input-Output-Paaren von Code-Beispielen trainiert, um dann für einen unbekannten Input neuen Code zu erzeugen (siehe die Literaturübersicht in [2]). Ein fortgeschritteneres Beispiel ist Tree2Tree [3]: bei diesem verfahren werden Syntaxbäume der Quellsprache in Syntaxbäume der Zielsprache übersetzt. Die wesentliche Herausforderung ist die Konstruktion eines geeigneten Aufmerksamkeitsmechanismus.

c) Die Nutzung von großen Sprachmodellen bietet sich an für die Übersetzung von Code von einer Programmiersprache in eine andere. Die wesentliche Differenzierung in diesem Ansatz findet statt zwischen Modellen, die speziell mit Quellcode trainiert wurden und solchen, die auf allgemeinen großen Datensätzen trainiert wurden.

3. Vorgehen und Erfahrungen

3.1 Verwendete Beispiele

Es wurden neben teilweise durch GenAI erzeugten kleinen Cobol- und PL1-Programmen im wesentlichen zwei Code-Beispiele für die Verprobung einer Portierung verwendet:

1. Ein bei einer Suche nach Cobolprogrammen im Internet gefundenes Cobol-Programm („Programm A“) mit einer Länge von etwa 5500 Zeilen, davon ca. 3000 Zeilen Definitionen von Variablen und Strukturen in der Data Division. Das Programm überprüft Input mit umfangreichen Eingabedaten anhand von etwa 30 Plausibilitätschecks, für die die Tabellen genutzt werden, und gibt Fehlerlisten und verschiedene aggregierte Daten auf dem Drucker aus. Es handelt sich nicht um ein Programm aus der Versicherungswirtschaft, aber es war als typischer Fall eines umfangreichen Cobolprogramms mit vielen hart codierten Daten nützlich. Aufgrund unbekannter Lizenzbedingungen wird das Programm hier nicht verlinkt.
2. Die General Insurance Application von IBM („IBM GenApp“), zu finden unter <https://github.com/cicsdev/cics-genapp> Es handelt sich um eine einfache Anwendung zur Verwaltung von Policen, Kunden und Schäden in Cobol, ohne Rechenkern, einschließlich Beispieldaten und Jobskripten. Länge des Cobol-Codes insgesamt ca. 10 Tsd. Codezeilen in 45 Dateien.

3.1 Vorgehen für die Portierung

Die Portierung erfolgte für die Tests nach Python und erwies sich als praktikabel. Teilweise wurde ebenfalls nach Java und C portiert, was ebenfalls funktionierte.

1. Vorgehen für die Portierung kleiner Codeteile: Durch ein LLM mit dem Prompt: „Erzeuge ein Python-Programm, das funktional voll äquivalent zum Code in der Datei X ist.“ Abhängig vom verwendeten Modell war entweder das erste Resultat bereits in Ordnung und lauffähig oder es brauchte Iterationen, die händisch oder mit agentic AI-Tools durchgeführt wurden.
2. Vorgehen für die Portierung größerer Codes: Hier wurden zwei Möglichkeiten verprobt:
 - a. Automatische oder händische Aufteilung des Codes in kleinere Teile, Portierung jeweils entsprechend Nummer 1, dann „Zusammenfügen“ des Gesamtcodes.

- b. Portierung des gesamten Codes mit Iterationen, ganz oder teilweise mit agentic AI.

3.2 Ergebnisse und Lessons learned

Schlussendlich konnten alle verprobten Cobol-Programme erfolgreich nach Python portiert werden. Die wesentlichen Erkenntnisse waren:

1. Eine "Portierung in einem Rutsch" erwies sich nur für kleine Codefragmente als verlässlich, wobei die Größe abhängig vom Modell war. Bei größeren Code-Teilen kam es abhängig vom verwendeten Modell z.T. auch bei vielen Iterationen nicht zu einer korrekten Portierung.
2. LLM-generierter Code kann fehlerhaft oder unvollständig sein - und ist es in der Regel am Anfang auch. Tests zur Validierung der Korrektheit sind erforderlich; deren Resultate können manuell oder bei Nutzung von AI-Agenten auch automatisch genutzt werden, um den Code in weiteren Iterationen zu korrigieren bzw. zu vervollständigen.
3. LLMs können nur eingeschränkt gut mit umfassenden Tabellen wie in Programm A, die de facto lange Listen von Parametern sind, umgehen: Die Kopie solcher Tabellen in eine andere Sprache ist oft unvollständig oder fehlerhaft. Es bewährt sich daher, diese Informationen anders einzubinden, z.B. indem man sie vorverarbeitet, z.B. die Variablennamen und Werte extrahiert und als Parameterdatei bereitstellt.
4. Die Portierung per LLM erlaubt auch Veränderungen des Codes entsprechend zusätzlichen Anforderungen/Spezifikationen. Das ist ein wesentlicher Unterschied zu klassischen Methoden der maschinellen Portierung. Bei den durchgeführten Beispielen wurde die Portierung genutzt, um zusätzliche Verbesserungen einzubauen:
 - a. alternative Ausgabeformate (JSON statt Drucklisten in Programm A),
 - b. Erstellung einer REST-API und darauf basierend einer einfachen Web-UI (IBM GenApp)
5. Der manuelle Aufwand hängt stark vom Modell, dem Ursprungscode, den zusätzlichen Anforderungen und den verwendeten Programmen zur Nutzung des Modells ab. Als Faustregeln gilt:
 - a. Erforderliche Iterationen können manuell oder teilweise automatisch durch agentische Workflows umgesetzt werden. Je höher der automatische Anteil und eine Absicherung beispielsweise durch im Workflow integrierte Tests, desto geringer der manuelle Aufwand.
 - b. Das verwendete Modell spielt eine große Rolle. Beispielsweise war eine Portierung mit GPT o3 und GPT o4mini sehr aufwändig mit vielen Iterationen, wurde aber von GPT 5 mit nur wenigen Iterationen bewältigt. Die Nutzung der jeweils neuesten verfügbaren Modelle lohnt sich in der Regel.

3.3 Dokumentation

LLMs können umfangreiche Codebasen abschnittsweise dokumentieren und so die Funktionsweise für heutige Entwickler nachvollziehbar machen. Dies ist insbesondere wertvoll, wenn Dokumentation fehlt oder lückenhaft ist. Es handelt sich dabei quasi um eine Portierung nach „Deutsch“. Ergebnisse und Lessons learned sind insofern ähnlich wie für

das Thema Portierung. Da der Anspruch an die semantische und syntaktische Korrektheit geringer ist als für Code, ist diese Portierung tendenziell leichter.

Um zu prüfen, ob ein LLM auch (ggf. bewusst) verkomplizierten Code dokumentieren kann, wurde ein Sortieralgorithmus (Quicksort) bewusst kompliziert implementiert. Die Dokumentation funktionierte ebenfalls.

4. Dokumentation der Ergebnisse

4.1 Repo https://github.com/abernert/dav_ag_ki

Im folgenden ein kurzer Überblick über die im Repo https://github.com/abernert/dav_ag_ki liegenden Informationen.

4.1.1 Portierte und dokumentierte IBM GenApp

Die GenApp von IBM wurde erfolgreich nach Python portiert. Das Ergebnis befindet sich im Repo. Dort befindet sich ebenfalls eine automatisch erzeugte Dokumentation / Spezifikation der Applikation. Diese wurde aus einer Version des Repo ohne die dort bereits befindlichen .md-Dateien durch eine agentic AI erzeugt.

4.1.2 Einfaches Agentic-AI-Programm zur Portierung

Ein im Rahmen der DAV Arbeitsgruppe entwickeltes Python-Skript / -Modul, VeriPort, zur Portierung von Code von einer beliebigen (vom LLM verstandenen) Programmiersprache in Code einer beliebigen anderen (vom LLM verstandenen) Programmiersprache. Es befindet sich im Repo.

Das Programm folgt einer einfachen Agentic-AI-Struktur bestehend aus einem Codierer und einem Verifizierer. Als agentisches Framework wird CrewAI verwendet. Der Ablauf ist wie folgt:

1. Ein „Converter“-Agent erzeugt Zielcode in der gewünschten Sprache.
2. Ein „Reviewer“-Agent vergleicht Original und Konvertat auf Funktionsgleichheit und Syntax.
3. Bei Ablehnung fließt das Feedback in die nächste Iteration ein (max. `--max-iters`).
4. Sobald der Review „approve“ liefert, schreibt VeriPort die konvertierte Datei auf die Platte.

Anmerkung: Dieses Skript eignet sich vorrangig zur Portierung kleinerer Dateien. Für größere Dateien und/oder mehrere Dateien sind fortgeschrittenere Programme und Agentic-AI-Strukturen erforderlich. Für diesbezügliche Fragen wende man sich an den Maintainer des Vericode-Repos.

4.1.3 Verkomplizierte Quicksort-Implementierung und Dokumentation

Das C-Programm kowabunga_spaghetti.c wurde als eine unnötig komplizierte Quicksort-Implementierung erstellt. Es konnte durch das LLM gut dokumentiert werden; das Ergebnis befindet sich ebenfalls im Repo.

4.2 Vortrag

Teile der Ergebnisse wurden überblicksweise u. a. auf der KI-Tagung von bf21 in 2025 vorgestellt; die zugehörigen Folien sind ebenfalls im Repo hinterlegt.

5. Fazit: Chancen und Grenzen

Chancen

1. Reduktion der Abhängigkeit von Legacy-Expert:innen,
2. kostengünstigere und flexiblere IT-Infrastruktur,
3. schnellere Dokumentation und Modernisierung,
4. Möglichkeit, Altsysteme schrittweise und mit deutlich geringerem Aufwand zu modernisieren bzw. durch eine neue vollständige und konsistente Dokumentation wieder wartbarer zu machen,
5. Erweiterungen (Schnittstellen, Formate) ohne komplettes Redesign.

Grenzen

1. LLMs benötigen sorgfältige Steuerung und iterative Korrektur, manuell oder über agentic AI,
2. fehlende oder unzureichende Testbasen erschweren die Absicherung,
3. Projekte erfordern weiterhin Fachwissen und Ressourcen; eine „One-Click-Lösung“ existiert bisher nicht. Dies gilt insbesondere im Fall von komplexen technischen Infrastrukturen, die mit den Applikationen „verschränkt“ sind.

Die Erfahrungen zeigen, dass LLMs eine tragfähige Alternative im Umgang mit Legacy-Systemen darstellen. Neben Weiterbetrieb und vollständiger Neuentwicklung gibt es nun die Option, bestehende Systeme gezielt in moderne Programmiersprachen zu übersetzen und dabei zugleich zu dokumentieren, zu erweitern und in neue Umgebungen zu integrieren.

Damit eröffnet sich insbesondere für Aktuarate eine realistische Möglichkeit, alte Rechenkerne zu erhalten und gleichzeitig technologisch zu modernisieren, ohne in Großprojekte mit vollständiger Neuentwicklung einsteigen zu müssen.

6. Literatur

[1] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pougeum Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In 2024 IEEE/ACM 46th International Conference on Software Engineering. <https://doi.org/10.1145/3597503.3639226>

[2] Xinyun Chen, Chang Liu, Dawn Song. 2018. Towards Synthesizing Complex Programs From Input-Output Examples. In International Conference on Learning Representations (ICLR), 2018.

[3] Xinyun Chen, Chang Liu, Dawn Song. 2018. Tree-to-tree Neural Networks for Program Translation. In 32nd Conference on Neural Information Processing Systems (NIPS 2018). <https://doi.org/10.5555/3327144.3327180>